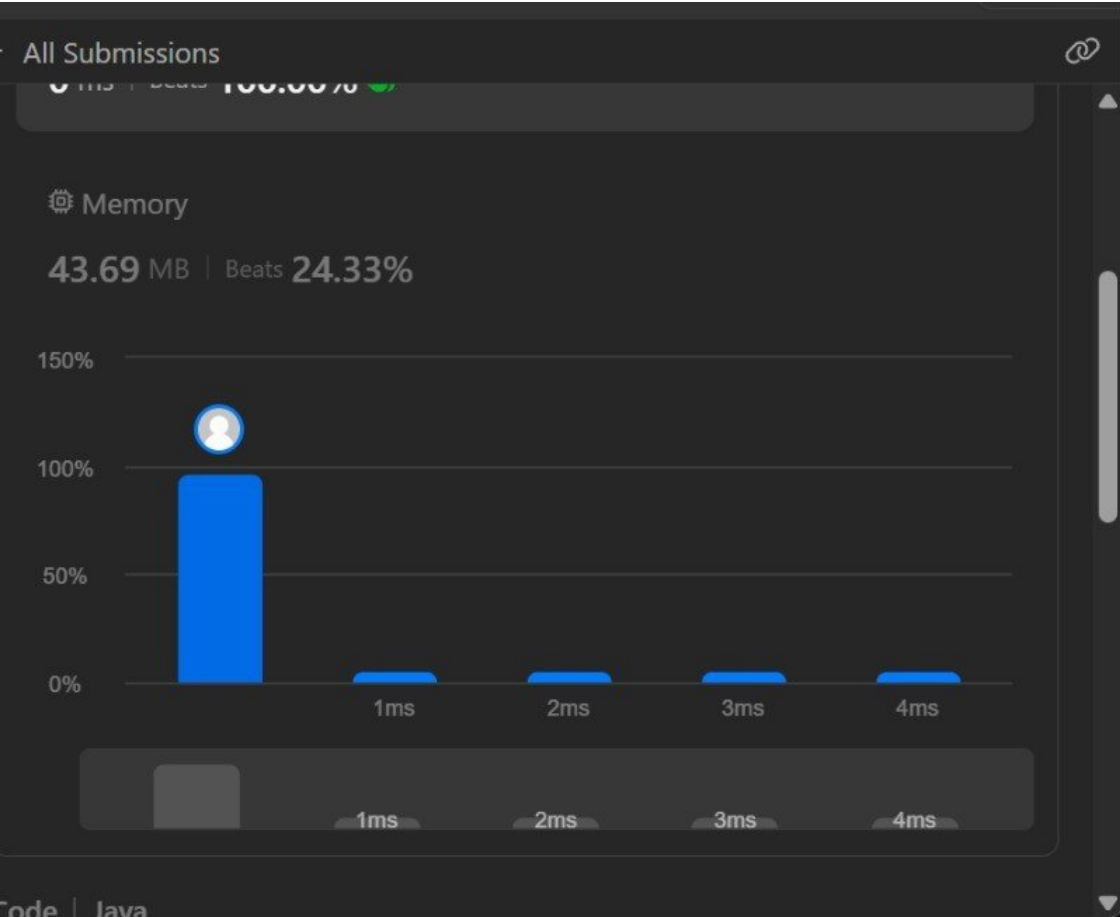




Java ▾ 🔒 Auto

```
25     Arrays.toString(nums2));  
26     // Output: k = 5, nums = [0, 1, 3, 0, 4, _, _, _]  
27 }  
28
```

Saved

Ln 28, C

☒ Testcase | ☒ Test Result

☒ Case 1 ☒ Case 2

Input

nums =

[3,2,2,3]

val =

Description | Accepted × | Editorial | Solutions | Submissior <

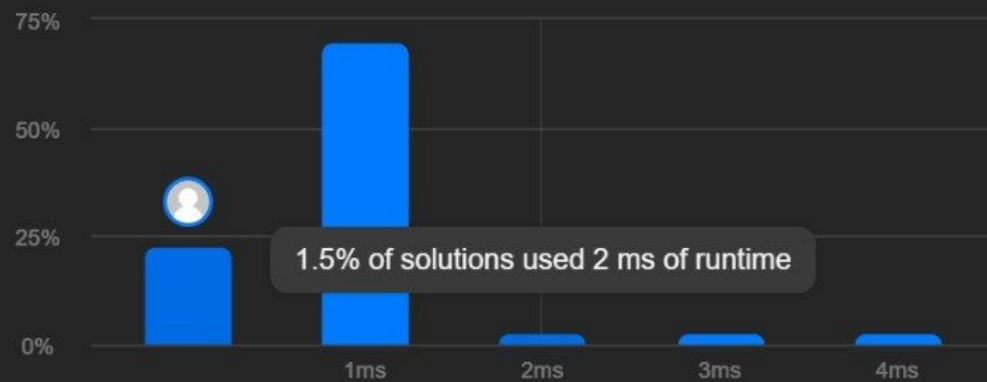
< All Submissions

Runtime

0 ms | Beats 100.00% 🏆

Memory

46.40 MB | Beats 96.24% 🏆



</> Code

Java ▾ Auto

```
28 |  
29 | }  
30 |
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

nums =
[1, 1, 2]

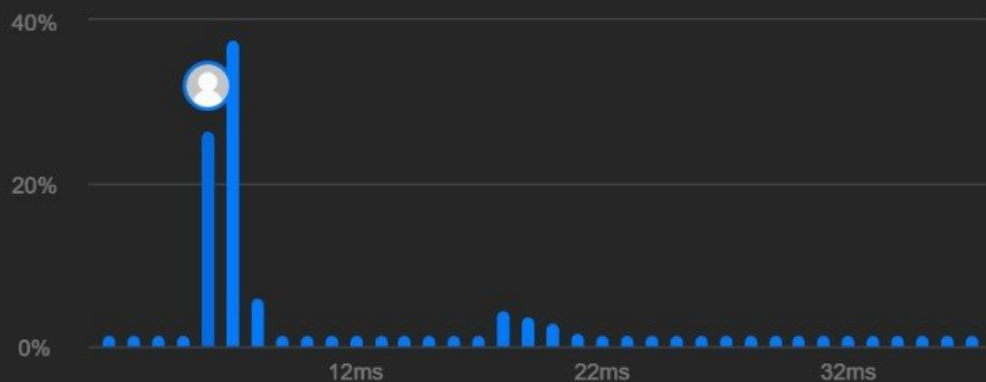
Output

Runtime

6 ms | Beats 99.29%

Memory

49.28 MB | Beats 96.22%



Code

Java Auto

```
27 System.out.println(sol.groupAnagrams(strs2)); // [[]]
28 System.out.println(sol.groupAnagrams(strs3)); // [{"a"}]
29 }
30 }
31
```

Saved

Ln 31,

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Input

strs =
["eat","tea","tan","ate","nat","bat"]

Output

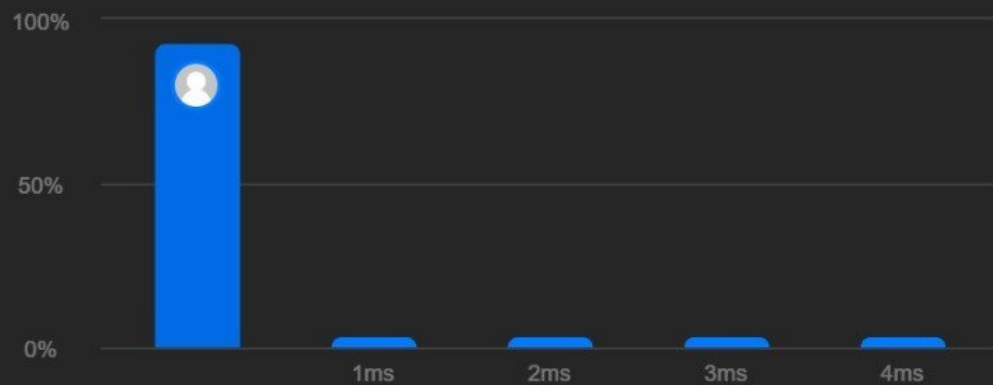
Description Accepted Editorial Solutions Submissions

All Submissions

0 ms | Beats 100.00%

Memory

43.23 MB | Beats 19.12%



1ms

2ms

3ms

4ms

Java

Auto

```
20 System.out.println(sol.largestAltitude(gain1)); // 1
21 System.out.println(sol.largestAltitude(gain2)); // 0
22 }
23 }
24
```

Saved

Ln 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

gain =
[-5,1,5,0,-7]

Description | **Accepted** × | Editorial | Solutions | Submission <

← All Submissions

1 ms | Beats 99.95% 🏆

 Memory

77.03 MB | Beats 83.07% 🙌

 Code

Java   Auto

```

23         System.out.println(sol.maxSubArray(nums2)); // 1
24         System.out.println(sol.maxSubArray(nums3)); // 23
25     }
26 }
27

```

Saved

Ln 27, C

☑ Testcase | ➤ Test Result

Accepted Runtime: 0 ms

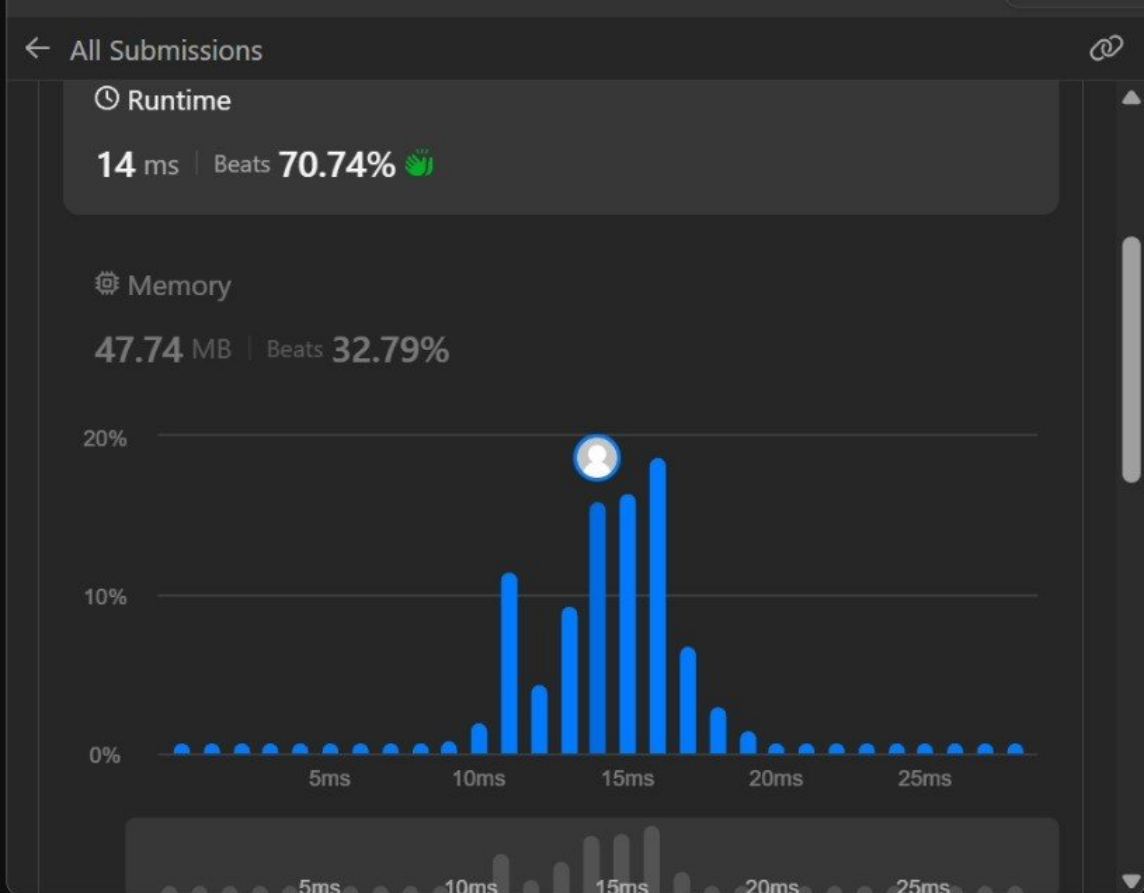
 Case 1

 Case 2

✔ Case 3

Input

```
nums =  
[-2,1,-3,4,-1,2,1,-5,4]
```



Java Auto

```
35     System.out.println(Arrays.toString(sol.topKFrequent(nums3,
36         2))); // [1, 2]
37 }
38
```

Saved Ln 38, Co

Testcase Test Result

Accepted Runtime: 1 ms

Case 1 Case 2 Case 3

Input

nums =

[1,1,1,2,2,3]

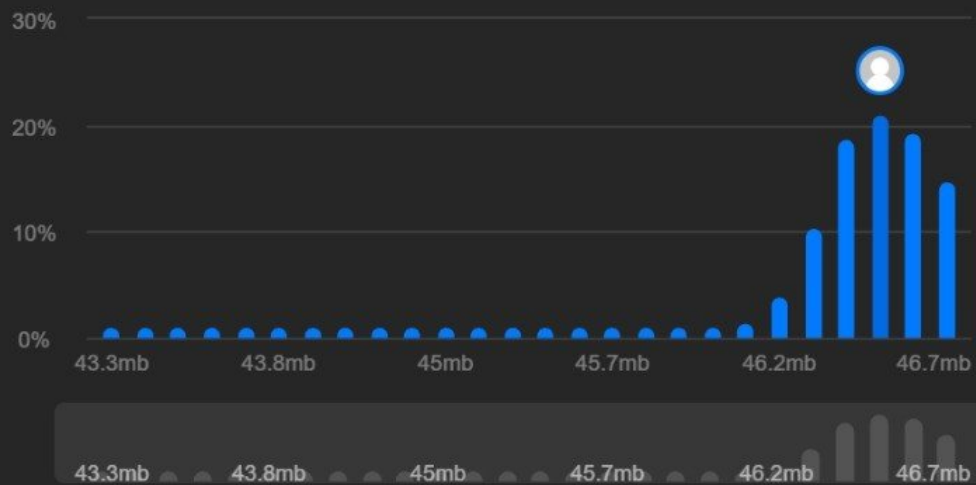
Description | Accepted x | Editorial | Solutions | Submissions

All Submissions

0 ms | Beats 100.00%

Memory

46.58 MB | Beats 41.68%



</> Code

Java Auto

```
23     System.out.println(java.util.Arrays.toString(sol.shuffle
24         (nums3, 2))); // [1, 2, 1, 2]
25     }
26 }
```

Saved

Ln 26, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Input

nums =

[2,5,1,3,4,7]

In computer science, a set is an abstract data type that can store certain values, without any particular order, and no repeated values(Wikipedia). $\{1, 2, 3\}$ is an example of a set, but $\{1, 2, 2\}$ is not a set. Today you will learn how to use sets in java by solving this problem.

You are given n pairs of strings. Two pairs (a, b) and (c, d) are identical if $a = c$ and $b = d$. That also implies (a, b) is not same as (b, a) . After taking each pair as input, you need to print number of unique pairs you currently have.

Complete the code in the editor to solve this problem.

Input Format

In the first line, there will be an integer T denoting number of pairs. Each of the next T lines will contain two strings seperated by a single space.

Constraints:

- $1 \leq T \leq 100000$
- Length of each string is atmost 5 and will consist lower case letters only.

Output Format

Print T lines. In the i_{th} line, print number of unique pairs you have after taking i^{th} pair as input.

Sample Input

```
5
john tom
john mary
john tom
mary anna
mary anna
```

Sample Output

```
1
2
2
3
3
```

Explanation

- After taking the first input, you have only one pair: (john,tom)
- After taking the second input, you have two pairs: (john, tom) and (john, mary)
- After taking the third input, you still have two unique pairs.
- After taking the fourth input, you have three unique pairs: (john,tom), (john, mary) and (mary, anna)
- After taking the fifth input, you still have three unique pairs.

39

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Line: 39 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1 5
2 john tom
3 john mary
4 john tom
5 mary anna
6 mary anna
```

Download

Your Output (stdout)

```
1 1
2 2
3 2
```

In computer science, a double-ended queue (dequeue, often abbreviated to deque, pronounced deck) is an abstract data type that generalizes a queue, for which elements can be added to or removed from either the front (head) or back (tail).

Deque interfaces can be implemented using various types of collections such as `LinkedList` or `ArrayDeque` classes. For example, deque can be declared as:

```
Deque deque = new LinkedList<>();
or
Deque deque = new ArrayDeque<>();
```

You can find more details about Deque [here](#).

In this problem, you are given N integers. You need to find the maximum number of unique integers among all the possible contiguous subarrays of size M .

Note: Time limit is 3 second for this problem.

Input Format

The first line of input contains two integers N and M ; representing the total number of integers and the size of the subarray, respectively. The next line contains N space separated integers.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq M \leq 100000$
- $M \leq N$

The numbers in the array will range between $[0, 10000000]$.

Output Format

Print the maximum number of unique integers among all possible contiguous subarrays of size M .

Sample Input

```
6 3
5 3 5 2 3 2
```

```
40
41
42
43
44
45
46
47
48
49
50
51
52
```

```
int count = map.get(removed);
if (count == 1) {
    map.remove(removed);
} else {
    map.put(removed, count - 1);
}
}

System.out.println(maxUnique);
in.close();
}
```

Line: 52 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Compiler Message

Success

Input (stdin)

1 6 3

2 5 3 5 2 3 2

Expected Output

1 3

Download

Download